GROUP MEMBERS:

1. SCM223-0222/2024 RAEL JUMA-GROUP LEADER

2. SCM223-1457/2024 FAITH ZUMA

3. SCM223-0205/2024 JACINTA MUTINDA

4. SCM223-1580/2024 IAN KIBERU

5. SCM223-1431/2024 BENTA YVONNE

6. SCM223-1454/2024 FESTUS MULEI

7. SCM223-0238/2024 VANESSA KERUBO

8. SCM223-1358/2024 IAN GITAU

STA 2240:FUNDAMENTALS OF OOP

STATISTICS 2.2 B

ASSIGNMENT(MILESTONE 3 & 4)

MILESTONE 3: Data Structures & Functional Abstraction

Objective

Milestone 3 focuses on transforming the system into a **data-driven processing engine** capable of handling real-world datasets efficiently.

1. Advanced Data Structures for Statistical Systems

- **Hash-Based Mapping (Dictionaries):** In a data-centric system, performance is dictated by lookup speed. We use dictionaries to map unique identifiers (like `app_id`) to game objects, ensuring $O(1)$ access time regardless of whether the Steam file has 1,495 rows or 1 million.
- **Set Theory for Data Integrity:** To handle categorical data like `genres` and `tags`, we utilize Sets. This allows the system to instantly identify unique categories and perform membership testing without the duplicates found in raw CSV strings.
- **Queues / Sequential structures** → represent processing stages
- **Lists** → store collections of data records

2. Functional Programming Paradigm

- **Immutability and Pure Functions:** We adopt a functional approach to data transformation. Instead of changing the raw Steam data, we use pure functions to return new, cleaned versions. This prevents "side effects" where data might be accidentally corrupted during the cleaning process.

- **Higher-Order Functions:** We implement `map` and `filter` operations.

  -Map: Used for element-wise transformations, such as converting the `estimated_owners` string ranges into numerical midpoints.

  -Filter: Used to create data subsets based on statistical criteria, such as isolating games with a `positive_reviews` count above a certain threshold.

3. File Handling and I/O Systems

This requirement moves your system from hard-coded "dummy data" to interacting with the physical filesystem.

- **Persistence Layer:** The system must read the raw CSV and write the resulting "cleaned" data back to disk in a structured format (CSV, JSON, or SQL).
- **Encapsulated I/O:** In OOP, I/O should be handled by a dedicated `DataManager` or `DataSource` class. This abstracts the complexity; the rest of your system shouldn't care *how* the file is read, only that it receives a stream of game data.
- **Encoding & Standards:** You must handle character encoding (like UTF-8) to ensure that international game titles in the Steam dataset are preserved during the read/write process.

4. Iterators, Generators, and Pipelines

This is the "Computational Thinking" heart of Milestone 3. It addresses how to handle 1,495 rows efficiently.

- **Iterators:** Objects that allow you to traverse through the game records one by one. This is essential for applying cleaning logic to every game in the dataset.
- **Generators (`yield`):** Instead of loading the entire 264.4+ KB dataset into RAM at once, a generator streams one row at a time. If your file grew from 1,495 rows to 1 million, a generator-based system would use the same amount of memory, whereas a standard list-based system would crash the program.
- **Pipelines:** You organize these generators into a sequence (e.g., `Load` → `Clean_Owners` → `Impute_Scores` → `Save`). Data flows through this "pipe" in a linear, predictable fashion.

5. Data Processing Workflows

This requirement defines the logical order of operations—the "Lifecycle" of a piece of data as it enters your system.

- **Ingestion:** The phase where raw strings from **"steam_top_games_2026 44.csv"** are brought into the system.
- **Transformation (Functional Abstraction):** Using higher-order functions like `map()` and `filter()` to perform calculations.

- **Validation:** Checking for the 955 missing `metacritic_score` entries and deciding how to flag them before they reach the analysis stage.
- **Workflow Orchestration:** The "Coordinator" object ensures that a game isn't analyzed before it has been cleaned, maintaining a strict logical sequence.

MILESTONE 4

1. Exception Handling Framework & Custom Classes

Data pipelines are prone to crashing when they encounter unexpected formats. Milestone 4 requires moving beyond simple `if/else` checks to a formal error-handling architecture.

- **Custom Exceptions:** Instead of using generic Python errors, you define domain-specific classes like `DataIntegrityError` or `MissingMetricError`.
- **Application:** When processing **steam_top_games_2026 44.csv**, your system might encounter one of the **7 missing release dates**. Rather than crashing, a custom exception is "caught," the incident is logged, and the system continues to process the remaining 1,488 records.

2. System Design Patterns

Design patterns provide a proven template for solving recurring architectural challenges. You must implement at least two:

- **Strategy Pattern:** This is perfect for the **955 missing Metacritic scores** in your dataset. You create interchangeable "strategies" for cleaning:
  - *Mean Imputation Strategy:* Fills the 955 gaps with the average score (approx. 77).
  - *Drop Strategy:* Removes games with missing scores from the analysis.
- **Observer Pattern:** You can implement a "Monitor" that "observes" the pipeline. Every time a record is cleaned or an error is caught, the Monitor updates a log file or a progress dashboard.
- **Decorator Pattern:** Use decorators to "wrap" cleaning functions to automatically log execution time without cluttering the actual cleaning logic.

3. Serialization and Data Storage

This requirement focuses on saving the "state" of your system so it can be resumed or audited.

- **Data Formats:** You implement serialization using formats like **JSON, Pickle, or CSV**.
- **State Persistence:** In a "Data Cleaning & Transformation Pipeline," serialization allows you to save a snapshot of the data *after* the owner ranges have been cleaned but *before* the Metacritic scores are imputed. This creates a "checkpoint" in your workflow.

4. Code Refactoring and Reuse

- **Refactoring:** You reorganize the code from Milestone 3 to eliminate redundancy and improve readability.
- **Modularity:** Logic for reading **steam_top_games_2026 44.csv** is separated from the logic used to clean it, allowing you to reuse the cleaning objects on other datasets in the future.